Understanding of Modern Software-Defined Networking

Phong Ngoc Hoang Nguyen
nhphong.nguyen@mail.utoronto.ca
University of Toronto

***Abstract*—This report summarizes an investigation of modern Software-Defined Networking (SDN), focusing on the evolution from the traditional network architecture, where the control plane and data plane are tightly coupled within individual network devices, to early SDN architecture that separate network control from packet forwarding, and ultimately to modern programmable network infrastructures deployed in large-scale data centers. The study examined ONOS, P4, Stratum, P4Runtime and Leaf-Spine Fabric. Particular attention was given to the separation of control and data planes, controller abstractions, programmable forwarding pipelines, and production-scale fabric architectures. The report concludes with some of the use cases, observations on the architectural tradeoffs and research challenges associated with programmable networks.**

***Index Terms*—Computer Network, Software-Defined Network, SDN**

I. Introduction

Started back in 1952, when IBM first manufactured large computer system and sold into the market as mainframe. It was bulky, large, and expensive machine that take up a room size. More importantly, every components within that mainframe is IBM's proprietary hardware, which means, a customer have to buy the whole vertically integrated solution from a single vendor regardless of the solution they want to have for their problem (software calculation, hardware design, finance...).

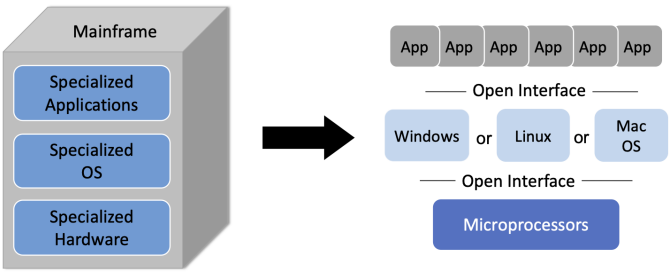


Fig. 1. Transformation from vertical market to horizontal marketplace with open interfaces at all level. Adapted from [1].

The evolution began with the arrival of open-source operating systems and microprocessors, which helped transform the market into a horizontal one where innovation could happen at any layer.

SDN initiatives draw the similar inspiration by decoupling the control plane from the data plane. Decision-making logic is decided by a external[1] logically centralized controller[2], using information from network operating systems enabled on top of the underlying switches. This abstraction simplifies switch's role, off-loading complex decision making logic (i.e., routing, path computation, ACL..) off-device and reducing the switch's primary task to packet forwarding. Such devices are therefore referred to as "white box" or "bare-metal" switches[3] which does not ship with a fixed network OS, creating a hardware ecosystem for multiple operating system vendors.

OpenFlow is the pioneer protocol that support this disaggregation in the early stage of SDN. Although it was a main instrumental propelling SDN into the mainstream, OpenFlow now play a small role of today SDN because of an architectural limitation it imposed—namely, its fixed-function match-action pipeline. Modern infrastructure has largely transitioned toward fully programmable data planes driven by the P4 programming language, P4Runtime, and Stratum. The objective of this study is to explore this architectural evolution from early OpenFlow to current P4-based ecosystems deployed across large-scale hyperscale data centers. Unless otherwise noted, the architectural exposition in the remainder of this report, including the accompanying figures, follows the treatment given in [1].

II. Basic Architecture

The core philosophy of Software-Defined Networking is horizontal disaggregation across every layers of the networking stack [1]. Rather than relying on a vertically integrated, propriety hardware appliances, SDN decompose them into abstract components that is independent from other components, allowing each of which being used interchangeably and innovated at each layer.

- **Local Network OS layer:** Physical switch hardware is decoupled from proprietary vendor software through

[1]In SDN terminology, a distinction exists between **external (global) network OS** and the **internal (local) network OS**. The External NOS referrs to a distributed controller platform to maintain a centralized view of the network (ONOS section) while Internal OS refers to lightweight OS running directly on-switch to manage internal switch operation and communicate with the external network OS.

[2]While SDN control plane is logically centralized to provide a unified global view of network state and simplify network management, production deployment (introduced in ONOS section) are physically distributed for high availability and scalability

[3]"bare-metal" switches run a ONIE bootloader uses facilities in Linux kernel and BusyBox, defining an open source "install environment" for end users to install target network OS [3]

bare-metal whitebox switches. Utilizing an Open Network Install Environment (ONIE) bootloader [3], the switch can load any compatible local Network OS (e.g., Stratum, SONiC) to handle local hardware initialization and peripheral management.

- **Control-to-Data Communication Interface:** The control plane and data plane interact over well-defined consumer/provider API endpoints using standard protocol (e.g. OpenFlow, gRPC/P4Runtime). This bidirectional interface allows controller to program forwarding rules while collecting real-time telemetry, link states, metrics, and packet from the data plane.
- **Device Driver & Pipeline Layer:** This layer resolves hardware heterogeneity by splitting device control into low-level driver abstractions and pipeline-mapping logic. Analogous to a hardware SDK, the *device driver* handles low-level chip management, translate control protocol messages (e.g., OpenFlow or P4Runtime) directly into chip memory while absorbing vendor-specific behavior mismatches. Sitting above the driver, the *pipeliner* maps generic, high-level *flow objectives* into the exact multi-table forwarding pipeline required by the underlying ASIC, freeing higher-layer application logic from needing to know the switch's internal table layout.
- **Data Plane layer:** Packet processing logic is completely decoupled from fixed-function hardware ASIC pipeline, allowing enterprise to implement custom header protocols on demand instead of waiting for switch vendor for the protocol implementation in their silicon chips. Using P4 language, packet parsing, header modifications, and forwarding behaviors can be programmed, updated, and recompiled directly on the hardware at runtime.

A. Single-switch perspective: The Software Stack

This includes an overview of a SDN stack with the involvement of a switch and controllers, highlight the top-to-bottom software stack being used in the SDN architecture. The main component include the Control App, Network OS, Switch OS, and P4 program–which used for the programmable switch. Figure 2 also introduced an exemplar of open-source software we used for this report (e.g. SD-Fabric, ONOS, Stratum), and protocol uses for communication between each layer.

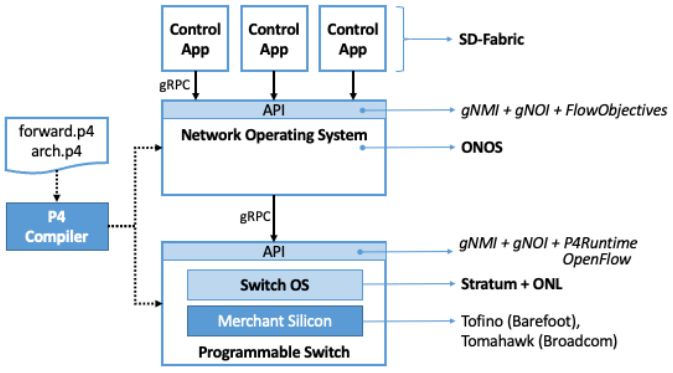


Fig. 2. Overall architecture of SDN stack with one switch. Adapted from [1].

At the top of the stack, Control Applications communicate with the Network OS via gRPC over Northbound APIs. Rather than writing raw table entries, applications issue high-level, topology-agnostic *Intent*, or hardware-agnostic routing intents called *Flow Objectives* (e.g., policy constraints or forwarding goals). Concurrently, management interfaces such as gNMI (gRPC Network Management Interface) and gNOI (gRPC Network Operations Interface) are exposed on the Northbound interface, enabling control applications to query dynamic network state (e.g., topology graphs, link statuses) and execute operational lifecycle commands (e.g., rebooting or upgrading devices).

Below the control abstraction is the separation of the control plane and the data plane. The Network OS compiles high-level Flow Objectives into low-level, device-specific *Flow Rules*—concrete match-action entries targeted at explicit hardware table from the entire hardware pipeline. These rules are pushed down via gRPC using southbound protocols such as P4Runtime, OpenFlow, or gNMI. The local Switch OS (e.g., Stratum, originally developed by Google) receives these rules and programs the corresponding entries into the hardware switch silicon[4]. Popular commercial implementations of this merchant silicon include Intel's Barefoot Tofino and Broadcom's Tomahawk chips.

B. Network-Wide perspective: The Entire Network Data Path

The previous subsection examined the SDN architecture through the lens of a single switch, a single controller, and its Network OS. This subsection zooms out to trace the data path end-to-end: from its origination at an end host, across one or more SDN-enabled switches, to its destination at another end host. As noted earlier, each switch along this path runs its own instance of a Switch OS, functioning as an on-host agent that programs flow rules onto the underlying silicon. The remainder of this subsection focuses on the end host itself, as the origination and termination point of the path; the inter-switch portion of the path—how multiple Switch OS instances are stitched together into a fabric under a common Network OS—is treated separately in Section V and the discussion of the Leaf-Spine Fabric.

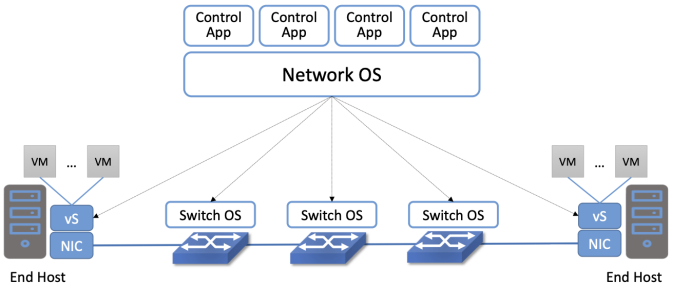


Fig. 3. End-End network view of SDN architecture. Adapted from [1].

[4]Switch silicon refers to a specialized integrated circuit (ASIC) designed for line-rate packet processing, switching, and hardware-accelerated QoS management.

With respect to the end host, Figure 3 highlights three components: the NIC, the virtual switch, and the VMs, all residing on the same physical server. A conventional NIC (Network Interface Card) is a hardware component that connects a computer or device to the network, converting data between the host's internal representation and the electrical or radio-frequency signals carried over the physical medium, and vice versa. The virtual switch (vSwitch, e.g., Open vSwitch) is a software switch that runs within the server's hypervisor and binds to the physical NIC as its uplink. Architecturally, the vSwitch occupies the same role as the Switch OS described in the single-switch view: it can be programmed by the same Network OS over southbound protocols such as OpenFlow or P4Runtime, receiving and installing Flow Rules exactly as a hardware Switch OS would. The distinction is one of substrate rather than function—the vSwitch executes its match-action pipeline in software on the host's general-purpose CPU [2], whereas a physical switch executes the equivalent pipeline on dedicated ASIC silicon. Once programmed, it behaves as an ordinary switch, receiving packets on an input port and forwarding them to the appropriate output port; the endpoints attached to these virtual ports are typically Virtual Machines (VMs) or containers.

That said, this architecture rests on a few assumptions worth making explicit. First, not every end host runs VMs; the same picture applies to containerized or bare-metal hosts, with the vSwitch's virtual ports simply terminating at a different type of endpoint. Second, the NIC–vSwitch–VM arrangement in Figure 3 is not the only valid way to structure the end host, but rather one particular perspective. This report, following [1], adopts a *network-centric* view, in which the vSwitch and NIC are treated as an extension of the network, controlled by the same Network OS as the rest of the fabric. A *host-centric* perspective is equally valid: it instead treats packet processing as a function of the host operating system itself, which means an architectural change in who is in control—instead of being managed by the network controller, it is configured directly by the host itself, via kernel-resident frameworks such as eBPF/XDP or user-space frameworks such as DPDK [1]. Third, the conventional NIC introduced above is itself only one option: vendors have a long history of offloading kernel functionality onto the NIC, and this has led to the Smart Network Interface Card (SmartNIC), a NIC augmented with an FPGA or ASIC capable of running programmable logic. A SmartNIC can replace the conventional NIC, and, going further, can assist or even replace the vSwitch entirely by implementing part or all of its forwarding pipeline directly in NIC hardware [1]. A more detailed treatment of these host-centric and SmartNIC-based alternatives, including their performance tradeoffs, is left for future work (Section **??**), as it requires background this report has not yet developed.

III. Programmable data plane and P4 language

Before turning to what P4 is and how the language itself is structured, it is worth establishing why a new data-plane programming model was needed at all. OpenFlow was built to reuse capability that switch vendors already had on-chip: McKeown et al. note that "we exploit the fact that most modern Ethernet switches and routers contain flow-tables (typically built from TCAMs)" used internally to implement functions such as firewalling, NAT, QoS, and statistics collection, and that OpenFlow simply exposes "this common set of functions" through a standard interface [4]. This was a deliberate compromise: rather than requiring new switch hardware, the design goal was a datapath "amenable to high-performance and low-cost implementations," which meant "forgoing the ability to specify arbitrary handling of each packet and seeking a more limited, but still useful, range of actions" [4]. In its original 2008 form this amounted to a single flow table matched against about a dozen header fields such as MAC address, IP address, protocol, and TCP/UDP ports [5].

A. The fundamental limitation of OpenFlow

That original dozen-field flow table did not remain fixed for long. The formal OpenFlow 1.0 specification followed the 2008 proposal in December 2009, and each successive revision of the OpenFlow 1.x line extended the matchable field set to accommodate new headers, so that the number of supported fields rose from 12 to 41 by 2013 [5]. Yet as data-center networking continued to evolve, operators kept demanding support for still newer encapsulation headers (e.g., VXLAN, NVGRE); rather than continuing to extend OpenFlow 1.x indefinitely, researchers proposed a strawman "OpenFlow 2.0" interface instead [5].

The push toward an OpenFlow 2.0 interface reflected two distinct pressures on any successor design: the need to keep pace with the rapid rate of change in networking requirements, and the need for a design that remained viable over the long term. The immediate problem was one of deployment speed. At the time, bare-metal switches still relied on a fixed-function pipeline hardcoded into the switch firmware to match packets, so supporting a new protocol required two slow steps in sequence. First, the protocol itself had to be agreed upon. Second, once agreed upon, the switch firmware itself had to be updated, since the firmware had no built-in intelligence to parse a raw packet format it had never been told about, and changing the firmware took still more time. Consequently, simply adding a new protocol to the specification was not enough on its own: it could still take vendors years to release a new switch model incorporating it, and for that hardware to then be adopted into the already-deployed network. The implication was that future switches should be able to define custom headers themselves, and that controller applications should likewise be able to understand these custom headers.

B. Existing efforts for programmable pipelines

Programmable forwarding pipelines existed well before P4, in several forms, each trading away something essential. Software switches and modular frameworks such as Click [7] offered complete flexibility, but executed on general-purpose CPUs and therefore operated far below the line rates expected of a switch ASIC; Click was also not designed for the

controller-switch split that SDN assumes, and so provided no notion of match-action tables populated at runtime by a controller [5]. At the other extreme, network processors, FPGAs, and—by 2013—reconfigurable match-action ASICs such as RMT [6] had demonstrated that this flexibility could in fact be achieved at terabit speeds. The obstacle there was not capability but usability: each chip exposed its own proprietary low-level interface, which Bosshart et al. compare to microcode programming, so a forwarding behavior written for one target could not be carried to another [5]. The gap P4 was created to fill is therefore a linguistic one rather than a hardware one: it provides a high-level, protocol-agnostic language that a compiler can translate into the configuration each target requires, including the line-rate programmable ASIC pipelines these chips had already made possible [5].

C. Flexible resource allocation

A further advantage of a programmable data plane over a fixed-function pipeline is in how on-chip memory is distributed among tables. In a fixed-function switch, both the set of supported protocols and the memory reserved for each table are fixed in silicon at design time, regardless of how a particular network actually uses them. A programmable pipeline instead treats its memory as a shared pool that the program can partition according to need, allowing table sizes to be matched to the deployment [6]. Fig. 4 illustrates this with a pipeline of four equal memory blocks. A fixed-function design dedicates one block each to the Ethernet, IPv4, IPv6, and ACL tables. If the network instead carries predominantly IPv4 traffic, a P4 program can declare an IPv4 table twice as large, occupying two blocks, while the IPv6 and ACL tables share the remaining one.

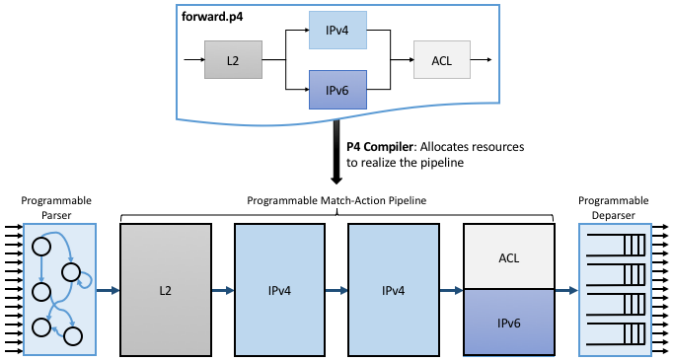


Fig. 4. Memory allocation across match-action tables in a fixed-function pipeline versus a programmable pipeline. Adapted from [1]

D. Forwarding pipeline

At the architecture level, researchers introduced PISA (Protocol Independent Switching Architecture) as the pipeline design that gives P4 its reconfigurability [1]. As shown in Fig. 5, a PISA pipeline is built from three components—a programmable parser, a programmable multi-stage match-action pipeline, and a programmable deparser—each configured by the P4 program itself rather than hardcoded into switch firmware. This generic, configurable structure is what lets the same abstract model describe packet processing across otherwise substantially different forwarding devices—Ethernet switches, load-balancers, and routers—and across different underlying technologies, whether a fixed-function switch ASIC, an NPU, a reconfigurable switch, a software switch, or an FPGA [5].

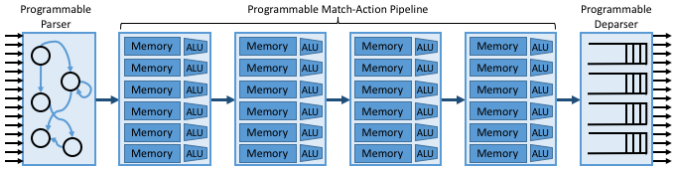


Fig. 5. Abstract forwarding pipeline that model how most data plane devices are forwarding packet. Adapted from [1].

The parser forms the first stage of the pipeline and extracts header fields from an incoming packet by walking a program-defined parse graph: it sequentially extracts each header and copies its values into the memory defined by that header's structure (e.g., the IPv4 header). Once extracted, these fields are passed to the match-action pipeline, where the design's protocol independence is most apparent. Each stage of the pipeline pairs a block of memory—combining SRAM and TCAM [1]—with an ALU. This memory is not preloaded by the switch vendor to recognize a fixed set of headers; that is, it does not inherently support IPv4 but not IPv6, or TCP but not UDP. It instead behaves as a generic bank of lookup-table entries that the P4 program itself populates at configuration time, using the match keys that a given header requires. The lookup is performed by the memory: the TCAM supports wildcard matching of the arriving packet's fields against the programmed entries, while exact matches are served from SRAM at lower cost [1]. Only after a match is found does the ALU execute the action associated with the matched entry. Because each stage can be pipelined—one stage processing one packet while the next stage concurrently processes another—this reprogrammable design still sustains line-rate throughput [1]. Finally, the deparser reassembles the modified header fields into a single serialized packet before it leaves the pipeline, following the header layout defined by the program.

E. Abstracting the pipeline

The pipeline described so far is a generic model, reduced to three components and abstracted away from any particular chip design. Production ASICs are considerably more elaborate and complex than this, and their pipelines differ from one vendor to the next, which raises the question of how a single program, or a single controller, can be written against such a variety of physical pipelines. The response is to interpose a single *logical pipeline*: an abstract model of the forwarding pipeline that programs and controllers are written against, and which a compiler, provided by the vendor themselves, then maps onto the physical pipeline of a given chip. With such

a model in place, controllers become pipeline-agnostic [1]. Fig. 6 illustrates this arrangement: the controller addresses a logical pipeline, and the P4 compiler maps that logical pipeline onto the differing physical pipelines of each ASIC beneath it.

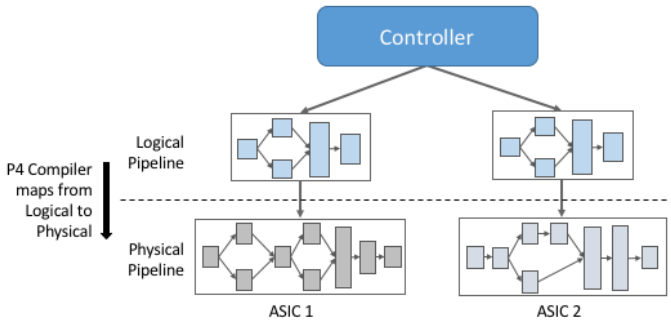


Fig. 6. A logical pipeline mapped by the P4 compiler onto the physical pipelines of two different ASICs. Adapted from [1].

The Portable Switch Architecture (PSA) is the standardization of this idea, developed by the P4 Consortium as a community effort [1]. It is intended to give P4 developers an abstract target machine, analogous to a Java Virtual Machine, and with it a write-once-run-anywhere programming model [1]. The industry, however, has not followed. Vendors do not provide the compiler backend that would map PSA onto their respective ASICs, which is precisely what the model depends on, and PSA therefore remains a mostly "on paper" artifact [1]. The incentives run the other way: vendors face their own version of the multi-ASIC problem as they release new chips over time, and defining an architecture of their own allows them to expose the distinctive capabilities of their silicon without being constrained by a standardization process [1]. What P4 developers write against in practice is therefore either V1Model, the simpler model illustrated in Fig. 7, or a vendor-specific architecture such as the Tofino Native Architecture (TNA), defined by Barefoot for its family of programmable switching chips [1]. In the latter case both the architecture and its backend compiler are proprietary, and the P4 program must conform to them. The portability objective is therefore not achieved: vendors retain the freedom to define their own pipelines, and a developer must write P4 against the architecture of the target in use. The resulting position nonetheless represents a significant advance on fixed-function forwarding. The pipeline is no longer determined in silicon by the vendor but is instead exposed to the developer, and it is this programmability, rather than portability across targets [1].

F. Example of a P4 program

The preceding subsections described the forwarding pipeline in the abstract; this subsection grounds that description in a concrete program. The example used throughout is `main.p4`, the P4 program written for this project, which is compiled for and executed on BMv2, the reference P4 software switch [10]. The program declares a range of header formats—Ethernet, IPv4, IPv6, TCP, UDP, ICMP, ICMPv6, and NDP—together

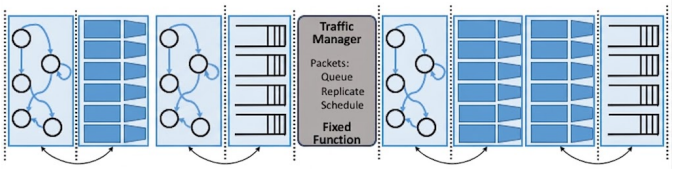


Fig. 7. V1Model. Adapted from [1].

with Segment Routing over IPv6 (SRv6) headers and a pair of CPU headers that carry packet-in and packet-out messages between the pipeline and the controller. The subsections that follow examine the program in the order a packet encounters it: the architecture it must conform to, the header formats it defines, the parser that extracts them, the tables that match on the extracted fields, and the actions those tables invoke.

1) The V1Switch pipeline: BMv2 executes programs written for the `v1model` architecture [10], whose `V1Switch` package fixes both the blocks a program must provide and the order in which they run: a parser, a checksum-verification block, an ingress pipeline, an egress pipeline, a checksum-computation block, and a deparser (Listing 1). A program cannot add, reorder, or omit a stage; a block it does not need must still be supplied with an empty body. These six are only the programmable stages. As shown in Fig. 7, the architecture also places a fixed-function traffic manager between the ingress and egress pipelines, which queues, replicates, and schedules packets [1]; it is not expressed in P4 and is directed only indirectly, through the metadata written during ingress.

```
V1Switch(
    ParserImpl(),
    VerifyChecksumImpl(),
    IngressPipeImpl(),
    EgressPipeImpl(),
    ComputeChecksumImpl(),
    DeparserImpl()
) main;
```

Listing 1. Instantiating the `V1Switch` pipeline in `main.p4`.

2) Header and metadata formats: P4 uses a C-like syntax for its type declarations, which makes this part of the program largely self-explanatory. A header is declared as an ordered sequence of named fields with explicit bit widths, and the program is free to define any header it requires; Listing 2 shows the Ethernet and IPv4 definitions. Widths may be given names of their own through `typedef`—for example `typedef bit<48> mac_addr_t`, naming the 48 bits that hold a MAC address. Values the program matches against are declared as named constants, such as `IP_PROTO_ICMP = 1` and `IP_PROTO_TCP = 6`, whose values are the protocol numbers assigned in RFC 790 [8]. Alongside the headers, the

program declares a metadata structure carrying the per-packet state the pipeline needs but that is not necessary to appear on the wire.

Not every header declared in this way corresponds to a protocol carried on the network. Listing 2 also defines `cpu_in_header_t`, which is prepended to packets sent to the controller over the CPU port and records the port on which the packet originally arrived. The `@controller_header("packet_in")` annotation identifies it as a control-plane header, from which the P4Runtime server populates the metadata fields of the corresponding packet-in message. A second header, `cpu_out_header_t`, serves the opposite direction, informing the pipeline of the port on which a packet received from the controller is to be transmitted. The declaration mechanism is therefore the same whether a header describes a standard protocol or a format defined solely for exchange between the pipeline and its control plane.

```
#include <core.p4>
#include <v1model.p4>

typedef bit<9>   port_num_t;
typedef bit<48>  mac_addr_t;
typedef bit<32>  ipv4_addr_t;

const bit<16> ETHERTYPE_IPV4 = 0x0800;
const bit<8> IP_PROTO_ICMP   = 1;
const bit<8> IP_PROTO_TCP    = 6;

header ethernet_t {
    mac_addr_t  dst_addr;
    mac_addr_t  src_addr;
    bit<16>     ether_type;
}

header ipv4_t {
    bit<4>   version;
    bit<4>   ihl;
    bit<6>   dscp;
    bit<2>   ecn;
    bit<16>  total_len;
    bit<16>  identification;
    bit<3>   flags;
    bit<13>  frag_offset;
    bit<8>   ttl;
    bit<8>   protocol;
    bit<16>  hdr_checksum;
    bit<32>  src_addr;
    bit<32>  dst_addr;
}

@controller_header("packet_in")
header cpu_in_header_t {
    port_num_t  ingress_port;
    bit<7>      _pad;
}
```

Listing 2. Header definitions in `main.p4`.

3) Packet parser: The parser is expressed as a finite state machine. Each state is introduced by the `state` keyword, and the transition out of that state is declared inside the state body itself.

```
parser ParserImpl (packet_in packet,
            out parsed_headers_t hdr,
            inout local_metadata_t l_metadata,
            inout standard_metadata_t metadata) {
state start {
    transition select(metadata.ingress_port) {
        CPU_PORT: parse_packet_out;
        default: parse_ethernet;
    }
}

state parse_packet_out {
    packet.extract(hdr.cpu_out);
    transition parse_ethernet;
}

state parse_ethernet {
    packet.extract(hdr.ethernet);
    transition select(hdr.ethernet.ether_type){
        ETHERTYPE_IPV4: parse_ipv4;
        ETHERTYPE_IPV6: parse_ipv6;
        default: accept;
    }
}
```

Listing 3. Parser state machine in `main.p4`.

Execution begins in the built-in `start` state; each subsequent state extracts one header from the packet into the header structure declared earlier, then transitions to the state responsible for interpreting the bytes that follow. The machine terminates in one of two built-in states: `accept`, marking the end of parsing and handing the extracted headers to the pipeline, or `reject`, marking a parsing failure [9].

4) Ingress processing: Ingress is the stage in which forwarding decisions are made. It is expressed as a set of match-action tables together with an `apply` block that sequences them. A table declares three things: the `key` it matches on, paired with a match kind such as `exact` or `ternary`; the set of `actions` that an entry may invoke; and the `default_action` taken when no entry matches. The table itself holds no entries in the program; these are installed at runtime by the controller, which is what allows forwarding policy to change without recompiling the pipeline. Listing 4 shows the L2 forwarding table and the action it invokes.

The `l2_exact_table` works like the MAC address table of a traditional switch: each entry maps a host's MAC address to the port the host is attached to. It exists so that frames between known hosts are forwarded directly. Once the controller learns where a host is and installs an entry for it, every frame addressed to that host goes straight out of the correct port, without being flooded to every port or sent to the controller.

```
action set_egress_port(port_num_t port_num) {
    standard_metadata.egress_spec = port_num;
}

table l2_exact_table {
    key = {
        hdr.ethernet.dst_addr: exact;
    }
    actions = {
        set_egress_port;
        @defaultonly drop;
    }
    const default_action = drop;
}

apply {
```

```
    if (!l2_exact_table.apply().hit) {
        l2_ternary_table.apply();
    }
    acl_table.apply();
}
```

Listing 4. An ingress table, its action, and the control flow that applies it.

Two properties of this arrangement are worth noting. First, an action may take parameters, such as `port_num` above, whose values are supplied by the matched entry at runtime rather than fixed in the program. Second, applying a table returns a result whose `hit` field can be tested, so the `apply` block expresses ordinary imperative control flow over the tables: here the exact-match table is consulted first, and the ternary table only if no entry matched. Forwarding is ultimately determined by writing `standard_metadata.egress_spec`, which the traffic manager reads once ingress completes.

5) Egress processing: Egress runs after the traffic manager, and therefore executes once per copy of the packet, at a point where the output port has already been fixed and is available read-only as `standard_metadata.egress_port`. Its role is consequently narrower than that of ingress: it applies adjustments that depend on the specific copy or on its destination port, as shown in Listing 5.

```
apply {
    if (standard_metadata.egress_port == CPU_PORT) {
        hdr.cpu_in.setValid();
        hdr.cpu_in.ingress_port = standard_metadata.
    ingress_port;
    }

    if (local_metadata.is_multicast == true &&
          standard_metadata.ingress_port ==
          standard_metadata.egress_port) {
        mark_to_drop(standard_metadata);
    }
}
```

Listing 5. Egress processing in `main.p4`.

The first condition handles packets destined for the controller: the `cpu_in` header is marked valid and stamped with the port on which the packet originally arrived, so that the controller receiving the packet-in message can recover that context. The second implements split horizon for multicast. Because replication has already occurred, one of the copies may be addressed to the port the packet arrived on; that copy is dropped, which prevents a broadcast NDP solicitation from being reflected back to its sender. Neither decision can be made during ingress, since neither the copy nor its egress port exists at that point.

6) Deparser: The deparser reassembles the outgoing packet. Its headers are passed as `in` parameters and are therefore read-only, and the only operation permitted is `emit`, so no further modification is possible at this stage [9]. The program specifies a single, static emission order, given in Listing 6.

```
control DeparserImpl(packet_out packet, in
    parsed_headers_t hdr) {
    apply {
        packet.emit(hdr.cpu_in);
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
        packet.emit(hdr.ipv6);
        packet.emit(hdr.srv6h);
        packet.emit(hdr.srv6_list);
        packet.emit(hdr.tcp);
        packet.emit(hdr.udp);
        packet.emit(hdr.icmp);
        packet.emit(hdr.icmpv6);
        packet.emit(hdr.ndp);
    }
}
```

Listing 6. The deparser in `main.p4`.

Although the order is fixed, the emitted packet is not. An `emit` on a header that is not valid has no effect, so only those headers that the parser extracted, or that the pipeline subsequently marked valid, appear on the wire. A single sequence therefore serializes packets of different shapes correctly: an IPv4 packet emits `ipv4` and skips `ipv6`, an IPv6 packet does the reverse, and a packet sent to the controller additionally carries the `cpu_in` header that egress marked valid.

G. P4 within the wider SDN stack

The preceding subsections treated the P4 program in isolation. In practice it is one component among several, and compilation is the point at which it is joined to the rest of the stack. The vendor supplies both the architecture and the backend compiler for its chips; the developer writes a program conforming to that architecture; and the compiler translates the program into a form the target can load. For the example used here, the program is compiled by the `p4c-bm2-ss` compiler, provided by the BMv2 community, against the `v1model` architecture. This compilation produces two distinct outputs.

The first, `bmv2.json`, is the target-specific configuration the software switch loads to realize the pipeline, and determines how packets are processed. The second, `p4info.txt`, is a Protocol Buffers description of the pipeline's control interface: match tables, match fields, match types, and the actions each table may invoke, with their parameters. It describes only the pipeline components that must be exposed to the control plane. Fig. 8 summarizes this workflow. Populating the pipeline using the P4Info is the role of P4Runtime, described in Section IV-B together with an example drawn from this program.

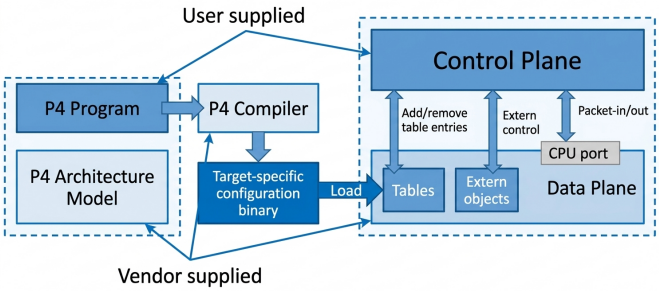


Fig. 8. P4 workflow.

Supplying the P4Info is not sufficient on its own. A controller that knows which tables exist still operates in generic, topology-agnostic terms, and something must map that intent onto entries in the specific tables this program defines. How the Network OS consumes these two artifacts, and what must be implemented alongside them, is the subject of Section V.

IV. Switch OS

This section turns to the software that runs on the switch itself. A defining property of the bare-metal switches introduced in Section II is that hardware and software are no longer sold as a single vertically integrated unit: using the ONIE bootloader, the operator can install any compatible Switch OS on any bare-metal switch, much as an operating system of choice is installed on a commodity server [1], [3].

The analogy to a server extends to the hardware itself. A Linux server always provides a general-purpose processor, with its registers, memory, and I/O, before any operating system is loaded onto it. A bare-metal switch is built the same way: it contains a conventional CPU on which the Switch OS runs, complemented by a switching chip that acts as a packet-forwarding accelerator [1]. The relationship between the two is closely analogous to that between a CPU and a GPU. The CPU runs the operating system and general-purpose software, and offloads a narrow but performance-critical workload to specialized hardware, here the forwarding of packets at line rate. The role of the Switch OS is therefore to expose the forwarding chip to the controller above it, receiving rules from the controller and installing them into the chip's tables. Beyond this, it performs only platform management, such as monitoring ports, transceivers, fans, and power supplies.

A. Switch OS architecture

The Switch OS considered here, Stratum, is described as a *thin* Switch OS [1]. It acts as a thin wrapper around the switch hardware API, exposing an interface through which a centralized SDN controller interacts with the hardware. Unlike the operating system of a conventional vertically integrated switch, it does not run complex, legacy control-plane protocol stacks locally; that logic resides in the controller.

On its northbound side, Stratum exposes three interfaces, each implemented as a gRPC service[5] [1]:

- **P4Runtime**, which controls the forwarding behavior of the switch by populating the tables defined in the P4 program;
- **gNMI** (gRPC Network Management Interface), which sets and retrieves the configuration of the switch;
- **gNOI** (gRPC Network Operations Interface), which manages the operational state of the switch.

For all three services, Stratum acts as the gRPC server and the Network OS as the client. The Network OS issues remote procedure calls against these services, and Stratum carries out each request on the switch hardware.

[5]gRPC is an open-source Remote Procedure Call (RPC) framework developed by Google for communication between services in data-center networks. Services and their messages are defined using Protocol Buffers.

By building on gRPC and Protocol Buffers, these interfaces inherit solutions to backward compatibility, security, data format, and reliable delivery, concerns that network protocols, OpenFlow included, have historically had to address individually [1].[6]

On its southbound side, Stratum relies on two vendor- and platform-specific components. The first is the Software Development Kit (SDK) supplied by the switch-chip vendor, which reads and writes the chip's specialized hardware and memory, analogous to a device driver in a conventional operating system. The second is the Open Network Linux Platform (ONLP) API, which provides access to the switch's peripherals, such as hardware counters, monitoring sensors, and status variables [1]. The remaining internal components of Stratum sit between these two sides and are designed to be vendor-agnostic, so that the same northbound interfaces are presented regardless of the chip or platform beneath them.

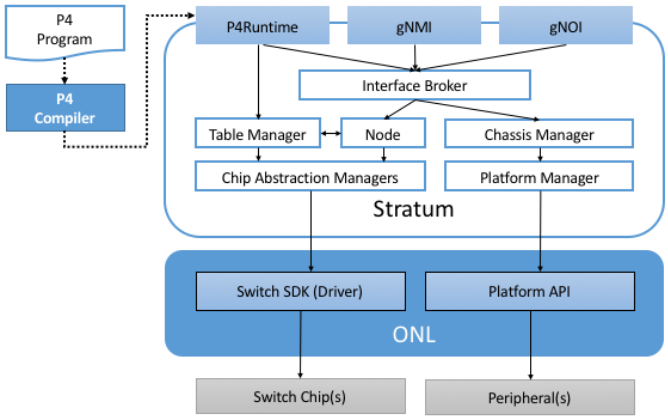


Fig. 9. "Thin" Switch OS architecture that is designed for vendor-agnostic. Adapted from [1].

B. P4Runtime

Section III-G ended with two files, `bmv2.json` and the P4Info, and an open question: how does a controller program tables it never defined? P4Runtime is the answer. P4Runtime is a control-plane protocol, built on gRPC and Protocol Buffers, through which a controller manages at runtime the data-plane entities of a device whose pipeline is defined by a P4 program, such as its tables, counters, and meters [11]. Every P4Runtime write has the same simple shape: put an entry in table X that matches value Y and runs action Z. The protocol itself does not know what X, Y, or Z mean; the P4Info tells the controller which tables, fields, and actions exist, and the controller picks the values. Because nothing about specific headers is built into the protocol, P4Runtime achieves protocol independence, just as P4 does. This is exactly what OpenFlow could not do (Section III-A), since its messages list the header fields a switch is allowed to match.

[6]Protocol Buffers define a language-neutral data format whose schema can evolve without breaking existing clients. gRPC secures each channel with TLS and runs over TCP, which provides reliable, ordered delivery, and uses HTTP/2 to multiplex concurrent streams over a single TCP connection.

Besides writing table entries, P4Runtime also supports [11]:

- Reading and updating other P4 objects, such as counters, meters, and action profiles.
- Multiple controllers per switch, so a backup can take over if the primary fails.
- Packet I/O, letting the switch send packets up to the controller and the controller send packets out through the switch.
- Batching many writes into one request.
- Loading a new P4 pipeline onto a switch that is already running.

Fig. 10 shows the overall workflow with P4Runtime. The P4 compiler turns the program into two outputs: a target-specific binary for the switch chip, and the P4Info, which is shared by both ends of the connection. The controller runs the P4Runtime client, the Switch OS runs the P4Runtime server, and the two communicate using the messages defined in `p4runtime.proto`. The figure shows backends for two hardware chips (Tomahawk and Tofino); in this project the BMv2 backend is used instead, and its target-specific output is `bmv2.json`. In practice, the workflow proceeds as follows:

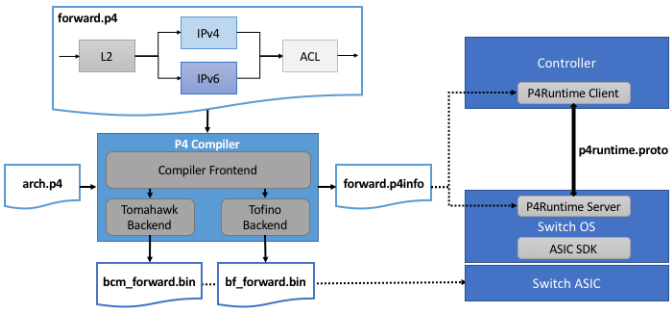


Fig. 10. The P4Runtime workflow. Adapted from [1].

1) The P4 program is compiled into `bmv2.json` and the P4Info, and both are given to the controller (ONOS in this project, see Section V).
2) When the switch boots, the controller connects to it and is elected as the controller in charge. The controller is the P4Runtime client; the Switch OS is the server.
3) The controller sends both files to the switch. The Switch OS loads `bmv2.json` into the forwarding chip, and keeps the P4Info so it understands the table and action IDs in later requests.
4) The controller starts writing table entries, and the switch begins forwarding traffic.

To make this concrete, Listing 7 shows the table `main.p4` uses for unicast bridging, and Listing 8 shows what the compiler wrote about it in `p4info.txt`.

```
action set_egress_port(port_num_t port_num) {
    standard_metadata.egress_spec = port_num;
}

table l2_exact_table {
    key = {
        hdr.ethernet.dst_addr: exact;
    }
    actions = {
        set_egress_port;
        @defaultonly drop;
    }
    const default_action = drop;
}
```

Listing 7. The `l2_exact_table` in `main.p4`.

```
tables {
  preamble {
    id: 33605373
    name: "IngressPipeImpl.l2_exact_table"
    alias: "l2_exact_table"
  }
  match_fields {
    id: 1
    name: "hdr.ethernet.dst_addr"
    bitwidth: 48
    match_type: EXACT
  }
  action_refs { id: 16812802 }
  action_refs {
    id: 16796182
    annotations: "@defaultonly"
    scope: DEFAULT_ONLY
  }
  const_default_action_id: 16796182
  size: 1024
}
actions {
  preamble {
    id: 16812802
    name: "IngressPipeImpl.set_egress_port"
    alias: "set_egress_port"
  }
  params {
    id: 1
    name: "port_num"
    bitwidth: 9
  }
}
actions {
  preamble {
    id: 16796182
    name: "IngressPipeImpl.drop"
    alias: "drop"
  }
}
```

Listing 8. The P4Info entries for `l2_exact_table` and its actions.

Notice that the P4Info says nothing about how the table works; it only gives the controller names, IDs, and sizes. The table is ID 33605373, its key is a 48-bit exact match on the destination MAC address, and the action `set_egress_port` is ID 16812802 with one 9-bit parameter. The `drop` action is marked `DEFAULT_ONLY`, so the controller cannot use it in a normal entry. This is all the controller needs to write entries into the table.

To install an entry, the controller uses the P4Info to build a P4Runtime message. Each table, field, and action is identified by its ID, and each value is encoded in bytes according to the bit width given in the P4Info: 48 bits for the MAC address and 9 bits for the port. The message is sent to the switch with the `Write` RPC. For a host with MAC address `00:00:00:00:00:1a` attached to port 3, the request looks like Listing 9. Further details on how the controller builds this

request in this project are given as an example in Section V-A.

```
updates {
  type: INSERT
  entity {
    table_entry {
      table_id: 33605373
      match {
        field_id: 1
        exact { value: "\x00\x00\x00\x00\x00\x1a" }
      }
      action {
        action {
          action_id: 16812802
          params {
            param_id: 1
            value: "\x03"
          }
        }
      }
    }
  }
}
```

Listing 9. The resulting P4Runtime `Write` request (simplified).

The `type` field tells the switch what to do with the entry: `INSERT` adds it, `MODIFY` changes its action, for example if the host moves to another port, and `DELETE` removes it. On the switch side, the Switch OS uses its copy of the P4Info to resolve the IDs and installs the entry into the forwarding chip. From then on, frames addressed to that host leave through port 3 without involving the controller. The controller can also call the `Read` RPC to fetch the table's current entries, which it uses to check that the switch still holds the rules it expects.

C. YANG and OpenConfig

YANG and OpenConfig are the groundwork on which the gNMI and gNOI interfaces of Section IV-D are built. To see why they are needed, it is useful to first consider how device state has traditionally been described. Historically, network devices exposed their state through a Management Information Base (MIB): a standardized dictionary of device variables, such as interface status, health indicators, and traffic counters, defined uniformly across devices and therefore independent of any particular vendor [1]. Used together with the Simple Network Management Protocol (SNMP), the MIB gave operators read access to this state for monitoring purposes. The shift to SDN, however, requires write access as well, since a controller must not only observe a device but also configure it. Meeting this requirement meant revisiting how the device data model itself is defined [1].

YANG is the modeling language that emerged to fill this role. A YANG model defines each device variable together with a human-readable description of its meaning, while remaining precise enough to be read and written programmatically. In this respect YANG plays a role similar to XML Schema (XSD) or JSON Schema: it defines the structure of the data. Unlike those schema languages, however, YANG is not tied to a single encoding. XSD describes only XML documents and JSON Schema only JSON documents, whereas data defined by a YANG model can be encoded as XML, JSON, or Protocol Buffers [1].

OpenConfig is an industry-wide standardization effort, driven by network operators that are driving to a convergence, vendor-neutral, programmable network model [12]. The effort has two parts. The first is a set of data models, written in YANG, that describe the configuration and operational state of common device features, such as interfaces, routing protocols, and platform components, in the same way on every vendor's equipment. The second is a family of modern RPC-based interfaces built on gRPC: gNMI for reading and writing configuration and state, gNOI for performing operational actions on the device, gRIBI (gRPC Routing Information Base Interface) for programming routing entries, and gNSI (gRPC Network Security Interface) for managing the device's security policies, such as certificates and access control. These interfaces are intended as alternatives to traditional management protocols such as SNMP and NETCONF[7].

Listing 10 shows an example of an OpenConfig model, `openconfig-interfaces`, which describes the network interfaces of a device. The model is shown in the YANG tree diagram notation, in which each line is a node of the data tree: `rw` marks configuration that a client can read and write, `ro` marks operational state that is read-only, `*` marks a list whose entries are identified by the key in brackets, and `?` marks an optional node. The model defines a list of interfaces keyed by name, and each interface keeps its configuration (`config`) separate from its operational state (`state`), a convention followed throughout the OpenConfig models.

```
module: openconfig-interfaces
  +--rw interfaces
     +--rw interface* [name]
        +--rw name
        +--rw config
        |  ...
        +--ro state
        |  ...
        +--rw hold-time
        |  ...
        +--rw subinterfaces
           ...
```

Listing 10. The base OpenConfig model for network interfaces (abridged). Adapted from [1].

The base model describes only the properties that are common to every kind of interface. Properties specific to a particular kind of interface are added by separate modules through *augmentation*, which attaches new nodes to an existing model without modifying it. Listing 11 shows the `openconfig-if-ethernet` module, which augments each interface with an `ethernet` container. This container holds Ethernet-specific configuration, such as the MAC address, port speed, and duplex mode, together with the corresponding state, such as the hardware MAC address and the interface counters.

```
module: openconfig-if-ethernet
  augment /oc-if:interfaces/oc-if:interface:
    +--rw ethernet
```

[7]NETCONF is an IETF protocol for installing and retrieving device configuration, encoded in XML and typically carried over SSH. YANG was originally designed as the modeling language for NETCONF.

```
       +--rw config
       |  +--rw mac-address?
       |  +--rw auto-negotiate?
       |  +--rw duplex-mode?
       |  +--rw port-speed?
       |  +--rw enable-flow-control?
       +--ro state
          +--ro mac-address?
          +--ro auto-negotiate?
          +--ro duplex-mode?
          +--ro port-speed?
          +--ro enable-flow-control?
          +--ro hw-mac-address?
          +--ro counters
             ...
```

Listing 11. The Ethernet augmentation of the OpenConfig interfaces model (abridged). Adapted from [1].

Technically, gNMI can operate on any YANG model, and in contrast, YANG is also not tied to either gRPC or gNMI; in Stratum, however, the two are used together, with gNMI exposing the switch's configuration and state through the OpenConfig models. Both interfaces are described next.

D. gNMI and gNOI

gNMI defines a small set of RPCs through which a client reads and writes the data tree described by the device's YANG models [13]. Listing 12 shows the four RPCs that gNMI defines:

- `Capabilities` returns the capabilities of the target: the YANG models it implements, the encodings it accepts, and the gNMI version it supports;
- `Get` retrieves a snapshot of data from the target;
- `Set` modifies data on the target;
- `Subscribe` opens a stream over which the target sends updates for the data that the client has subscribed to.

```
service gNMI {
    rpc Capabilities(CapabilityRequest)
        returns (CapabilityResponse);
    rpc Get(GetRequest) returns (GetResponse);
    rpc Set(SetRequest) returns (SetResponse);
    rpc Subscribe(stream SubscribeRequest)
        returns (stream SubscribeResponse);
}
```

Listing 12. The RPCs defined by the gNMI service.

gNOI complements gNMI by covering operations that are not naturally expressed as reading or writing a value in the data tree, such as rebooting the switch, testing reachability, installing certificates, or transferring files [14]. Unlike gNMI, gNOI does not rely on YANG models: each operation is defined directly as an RPC in Protocol Buffers, and related operations are grouped into separate services, such as `System`, `Cert`, and `File`. Listing 13 shows part of the `System` service.

```
service System {
    rpc Ping(PingRequest)
        returns (stream PingResponse) {}
    rpc Traceroute(TracerouteRequest)
        returns (stream TracerouteResponse) {}
    rpc Reboot(RebootRequest)
        returns (RebootResponse) {}
    // ...
}
```

Listing 13. Part of the gNOI `System` service.

V. Network OS

A. From an application to a P4Runtime write

Section IV-B showed the P4Runtime `Write` request that installs a host's entry in `l2_exact_table` (Listing 9). Listing 14 shows where that request comes from. When ONOS discovers a new host, the `L2BridgingComponent` application builds a flow rule for `l2_exact_table`: match the host's MAC address on `hdr.ethernet.dst_addr`, and run `set_egress_port` with the port the host is attached to. The application refers to everything by the names that appear in the P4Info, not by numeric IDs.

```
String tableId = "IngressPipeImpl.l2_exact_table";

PiCriterion match = PiCriterion.builder()
    .matchExact(
        PiMatchFieldId.of("hdr.ethernet.dst_addr"),
        host.mac().toBytes())
    .build();

PiAction action = PiAction.builder()
    .withId(PiActionId.of(
        "IngressPipeImpl.set_egress_port"))
    .withParameter(new PiActionParam(
        PiActionParamId.of("port_num"),
        port.toLong()))
    .build();

FlowRule rule = Utils.buildFlowRule(
    deviceId, appId, tableId, match, action);
flowRuleService.applyFlowRules(rule);
```

Listing 14. Building an `l2_exact_table` entry in `L2BridgingComponent.java` (simplified).

ONOS then uses the P4Info to turn this rule into that `Write` request. Each name is replaced by its ID, and each value is encoded in bytes according to the bit width given in the P4Info: 48 bits for the MAC address and 9 bits for the port. The application therefore never deals with numeric IDs or byte encodings; it only uses the names defined in the P4 program.

VI. Leaf-spine Fabric

References

[1] L. Peterson, C. Cascone, B. O'Connor, T. Vachuska, and B. Davie, "Software-Defined Networks: A Systems Approach," Systems Approach, LLC. [Online]. Available: https://sdn.systemsapproach.org/intro.html. Accessed: Sep. 5, 2026.

[2] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, "The Design and Implementation of Open vSwitch," in *Proc. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15)*, Oakland, CA, USA, May 2015, pp. 117–130.

[3] Open Compute Project, "Open Network Install Environment (ONIE) Overview." [Online]. Available: https://opencomputeproject.github.io/onie/overview/index.html. Accessed: Sep. 6, 2026.

[4] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, Apr. 2008.

[5] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, Jul. 2014.

[6] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, and M. Horowitz, "Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN," in *Proc. ACM SIGCOMM 2013*, Hong Kong, China, Aug. 2013, pp. 99–110.
[7] E. Kohler, R. Morris, B. Chen, J. Jannotti, and M. F. Kaashoek, "The Click Modular Router," *ACM Transactions on Computer Systems*, vol. 18, no. 3, pp. 263–297, Aug. 2000.
[8] J. Postel, "Assigned Numbers," RFC 790, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/info/rfc790/. Accessed: Sep. 20, 2026.
[9] The P4 Language Consortium, "$P4_{16}$ Language Specification." [Online]. Available: https://p4.org/specs/. Accessed: Sep. 20, 2026.
[10] The P4 Language Consortium, "behavioral-model: The reference P4 software switch (bmv2)." [Online]. Available: https://github.com/p4lang/behavioral-model. Accessed: Sep. 18, 2026.
[11] The P4.org API Working Group, "P4Runtime Specification." [Online]. Available: https://p4lang.github.io/p4runtime/spec/main/P4Runtime-Spec.html. Accessed: Oct. 9, 2026.
[12] OpenConfig, "OpenConfig." [Online]. Available: https://www.openconfig.net/. Accessed: Oct. 11, 2026.
[13] OpenConfig, "gRPC Network Management Interface (gNMI) Specification." [Online]. Available: https://github.com/openconfig/reference/blob/master/rpc/gnmi/gnmi-specification.md. Accessed: Oct. 12, 2026.
[14] OpenConfig, "gRPC Network Operations Interface (gNOI)." [Online]. Available: https://github.com/openconfig/gnoi. Accessed: Oct. 12, 2026.